Python Boundary Element Method Program for Underground Elastodynamics

Introduction to the Python Translation and Validation Program

Translation of Konstantinos Kostoglou's PhD boundary-element program from Fortran to Python

This document describes the Python translation of Konstantinos Kostoglou's PhD boundary-element program, its current capabilities, example inputs, validation evidence, and known limitations. The application retains the original Fortran source for comparison while providing a user friendly Graphical User Interface (GUI), DAT execution, saved results, and validation notes.

1 Purpose and background

The program uses the boundary element method (BEM) to analyse two-dimensional elastodynamic response of underground structures and cavities subjected to plane waves and explosion loading. It retains the governing numerical methods of the PhD program while presenting them through Python modules, a graphical input form, command-line execution, DAT examples, and documented comparisons.

The translation is also an improvement project. The original input decks and reference outputs are preserved for traceability, while the Python implementation adds GUI, a clearer data structures, reusable workflows, figure-numbered test decks, result archives, CSV export, and a documented validation process.

2 Physical capabilities

Capability	Description
P waves	Longitudinal plane-wave loading and P-wave propagation cases.
SV waves	Transverse shear-wave loading and the associated boundary response.
Combined P and SV waves	Cases in which longitudinal and transverse components act together.
Point explosions	Incident loading from an explosion at a specified coordinate, including supported decay and coupling parameters.
Underground structures	Cavities, liners, rectangular openings, half-spaces, free surfaces, and multi-zone configurations.
Dynamic response	Laplace-domain and supported direct time-domain response, including displacement, traction, stress, and deformation results.

Representative thesis inputs are retained as In-*.DAT files, with their validation status recorded separately.

3 Program modules

Module area	Main role
Input deck	Reads DAT files, material data, sections, zones, wave loads, explosion loads, steps, and plotting selections.
Geometry and mesh	Creates line elements, circles, rectangles, cavities, normals, connectivity, and boundary node types.

Module area	Main role
Kernels	Evaluates elastostatic, Laplace-domain, reflection, and time-domain fundamental solutions.
Loads	Builds P, SV, combined-wave, and point-explosion incident stresses and tractions.
Assembly	Forms boundary matrices and history kernels for the supported workflows.
Solver and transforms	Solves mixed boundary systems, traction histories, Laplace spectra, and inverse transforms.
Multi-zone workflow	Preprocesses zones and interfaces and assembles coupled systems.
GUI and plotting	Provides case selection, editable inputs, DAT browsing, result viewing, node plots, and CSV export.
Fortran I/O	Reads retained historical binary/text result formats for comparison.

4 How a calculation runs

1. Select a case from the Example list or browse to a DAT input file.
2. Parse the title, domain, geometry, materials, load records, time settings, and output selections.
3. Generate the boundary mesh and classify nodes and boundary conditions.
4. Construct incident P, SV, combined-wave, or explosion loading.
5. Assemble the appropriate Laplace-domain or time-domain boundary system.
6. Solve for displacements and tractions, then calculate available stress quantities.
7. Save the result as an NPZ archive and display plots or export CSV data.

5 Running the program

Graphical interface

From the BEM-underground project directory, run RUN_BEM_GUI.cmd. Select a DAT example from .\examples\in using Browse DAT, or edit the visible input cells as required.

The .\examples\in\Prev1992 directory contains the original PhD examples for reference. These have been superseded by the verified DAT examples in .\examples\in.

Press RUN to calculate the response.

Existing NPZ results can be opened using Open existing result. Displayed data can be exported as a CSV file or PDF report.

Command line

Use the project Python environment for numerical execution. From PowerShell in the BEM-underground directory:

```
$env:PYTHONPATH = 'src'; & '.\.conda-env\python.exe' -m python examples\in\In-5.27-LD.DAT --output examples\results\npz\In-5.27-LD_result.npz
```

6 DAT examples and PhD validation

Maintained DAT inputs are in examples\in; archived 1992 decks are in its Prev1992 subdirectory. Python solver results are in examples\results\npz, and PDF reports are in examples\results\pdf. The optional examples\in\manifest.toml lists a subset of DAT inputs; the GUI's Example selector is not limited to that list.

Example	Use
In-5.27-LD.DAT	Chapter 5 semi-infinite lined cavity; corrected ground-surface classification
In-5.31-LD.DAT	Chapter 5 two steel-lined cavities; compare LD and direct TD with the thesis
In-6.18-LD.DAT	Chapter 6 half-space P wave with free-surface reflection
In-6.20-LD.DAT	Chapter 6 reflected explosion input; Python solver support remains incomplete
In-6.22-LD.DAT	Chapter 6 discretized-ground explosion alternative; runnable
THESIS_RECT_EXPLOSION.DAT (Prev1992)	Historical rectangular point-explosion input

7 Testing and results

The runtime application is in src\python. The former examples\in\py test and comparison-script directory is absent from this checkout. Normal GUI calculations do not require it, but the former pytest and DAT inventory checks cannot be run until those scripts are restored. The references in pyproject.toml to that test path are historical.

Use the application smoke checks in the installation guide below. For numerical changes, restore the test scripts from a known project backup and rerun the corresponding validation comparisons before claiming thesis agreement.

Results are saved as NPZ archives with time values, mesh coordinates, displacements, tractions, and available boundary stresses. The validation reports compare selected computed values with mapped PhD figures; a scanned graph alone does not provide an exact numerical reference.

8 Current scope and future development

The implementation includes single-zone Laplace analysis, P and SV plane waves, point-explosion loading, multi-zone workflows, direct time-domain cases (DOMAIN 2), and a separate convolution-quadrature option (DOMAIN 5). Results can be inspected in the GUI or exported as CSV and PDF. A completed run does not by itself establish agreement with a PhD figure.

Remaining work includes direct-time enclosing-boundary equations, reflected explosion loading, and specialized legacy records. The In-6.20-LD and In-6.20-TD inputs specify the Chapter 6 reflected-explosion problem, but the current Python runner cannot solve them. The separate In-6.22-LD and In-6.22-TD inputs use the supported discretized-ground method. Chapter 5 LD and TD agreement also varies by case: the corrected In-5.27 LD peaks closely match the thesis, while direct TD retains a smaller peak; In-5.31 LD agrees more closely with the thesis displacements than direct TD. See validation\FIGURE_527_FINDINGS.md and validation\FIGURE_531_FINDINGS.md.

9 Reference files

- USER_GUIDE.md - practical installation and operating instructions.
- examples/README.md - example input and result locations.
- examples/in/manifest.toml - machine-readable list of selected DAT cases.
- Documentation/TRANSLATION_GAPS.md - remaining translation scope.
- validation/thesis_figure_runs/VALIDATION.md - figure-run validation report.

10 Development environment and translation method

The conversion and continued development were carried out in Microsoft Visual Studio Code, with the ChatGPT Codex artificial-intelligence add-on installed as a development assistant. Codex supported the work across source inspection, Fortran-to-Python translation, test creation, input-deck analysis, numerical debugging, documentation, and validation organization.

The original program contains a large body of specialized numerical logic, historical input conventions, and interdependent routines. Codex made it possible to trace these relationships, translate them incrementally, compare them with the retained Fortran source and PhD figures, and repeatedly test the resulting implementation. Completing this conversion manually, while preserving the historical behavior and adding the new Python interfaces, would most probably have been impractical within a reasonable development effort.

11 Coverage of PhD application figures

The available In-*.DAT inputs and validation reports represent the main PhD application groups, including the following:

- Figures 2.1–2.4: the first infinite-plane-wave application group.
- Figures 2.5–2.8: the equal-element plane-wave comparison.
- Figures 2.9–2.15: the unequal-element plane-wave comparison.
- Figures 2.16–2.19: the rectangular or unequal mesh geometry application.
- Figures 2.20–2.23: SV-wave loading.
- Figures 2.24–2.27: combined P and SV loading.
- Figures 2.28–2.31: Laplace-domain point-explosion applications.
- Figures 2.32–2.39: additional explosion application groups retained from the historical decks.
- Figures 2.40–2.43: the rectangular cavity loaded by a point explosion.
- Figures 3.2–3.5, 3.6–3.9, and 3.10–3.13: Chapter 3 plane-wave and combined-wave time or transient application groups.
- Figures 3.14–3.20a: Chapter 3 time-domain explosion application groups.

Current examples include In-5.10a-LD.DAT, In-5.13-TD.DAT, In-5.27-LD.DAT, In-5.31-TD.DAT, In-6.18-LD.DAT, and In-6.22-TD.DAT under examples\in. Historical inputs such as THESIS_RECT_EXPLOSION.DAT remain under examples\in\Prev1992. Consult the validation notes before treating a computed result as a reproduction of a thesis curve.

12 Clarification of figure coverage

Figure-numbered inputs cover selected plane-wave, SV-wave, combined-wave, explosion, half-space, and multi-zone cases. The corresponding reports should be assessed against the recorded validation status. In particular,

the reflected-explosion Figure 6.20 inputs are not yet executable in Python, and some Chapter 5 direct TD results still differ from the PhD plots.

13 Python installation and environment

The checked-in launcher RUN_BEM_GUI.cmd expects a Windows Python environment at BEM-underground\.conda-env. This checkout currently has Python 3.13 and the required numerical and graphical packages there. On another computer, install Miniconda or Anaconda, open PowerShell in the BEM-underground directory, and create the environment from environment.yml. The project requires Python 3.13 or newer; the environment file pins Python 3.13.

```
conda env create --prefix .\.conda-env --file environment.yml
```

Check the interpreter and imports before opening the GUI. The second command verifies NumPy, SciPy, Matplotlib, and Tkinter without changing the project. A working Windows desktop is needed to display the GUI.

```
& '.\.conda-env\python.exe' --version
& '.\.conda-env\python.exe' -c "import numpy, scipy, matplotlib, tkinter; print('Python environment OK')"
```

Then run .\RUN_BEM_GUI.cmd. It sets PATH and PYTHONPATH for the repository's src\python package. If Conda is unavailable, create a Python 3.13 virtual environment and install the project dependencies with pip; launch the GUI manually because the CMD file specifically points to .conda-env.

```
py -3.13 -m venv .venv
& '.\.venv\Scripts\python.exe' -m pip install -e .
$env:PYTHONPATH = 'src'; & '.\.venv\Scripts\python.exe' -m python.gui
```

14 DAT files and choosing a case

Open maintained inputs in examples\in. The In-*.DAT names identify PhD figure cases; paired -LD and -TD files select Laplace-domain and direct time-domain calculations. Files with -CQ select convolution quadrature. The Prev1992 subfolder retains historical decks and should remain separate from maintained examples. The manifest.toml file records a subset for reference; it is not an exhaustive list or a separate GUI category.

A DAT deck records the problem title, DOMAIN, boundary geometry and element counts, material properties, load, number of steps, total time, and requested plots. Multi-zone cases also include ZONES records. DOMAIN 1 is Laplace, DOMAIN 2 is direct time, and DOMAIN 5 is the separate CQ route. Select an existing case through the GUI's Example list or Browse DAT button. Copy a deck to a new name before changing materials, mesh, time step, or plot nodes; node numbers depend on the mesh.

For example, In-5.27-LD.DAT describes a lined half-space cavity with three boundaries, two zones, 20 steps over 0.015 s, and selected hoop stress nodes. Inspect TITLE, TYPE.PROBLEM, SECTION.TYPE, MATERIAL.PROPERTIES, STEP.NUMBER, TIME.TOTAL, LOAD.STRESSES, ZONES, PLOTS, and NODE.PLOTS before running a modified copy.

Run a DAT from PowerShell

From the BEM-underground directory, set the package path and choose a new output name:

```
$env:PYTHONPATH = 'src'
& '.\.conda-env\python.exe' -m python 'examples\in\In-5.27-LD.DAT' --output 'examples\results\npz\In-5.27-LD_check.npz'
```

A successful run prints the number of saved time steps. Some preserved input decks describe methods that have not yet been translated. In particular, In-6.20-LD.DAT and In-6.20-TD.DAT request reflected explosion loading and currently fail in the Python runner. Use the separate In-6.22 discretized-ground cases for the supported alternative.

15 Results reports and validation

Solver archives are stored in examples\results\npz. Open an existing NPZ in the GUI to inspect results without rerunning the calculation. PDF reports are stored in examples\results\pdf, while older report versions and other comparison artifacts remain under examples\results. The examples\reference folder holds the thesis-figure index and Appendix F DAT guidance. The original Fortran disks are preserved under src\fortran-original.

In the GUI, use the result quantity, node selection, and time-step controls to inspect displacement, traction, and available stress plots. Export CSV for numerical review or create a PDF report for a selected case. A displayed curve is a computed output, not proof of PhD agreement. Read validation\FIGURE_527_FINDINGS.md, validation\FIGURE_531_FINDINGS.md, and Documentation\TRANSLATION_GAPS.md for known comparisons and gaps.

16 Troubleshooting and maintenance

If RUN_BEM_GUI.cmd cannot find Python, recreate .conda-env from environment.yml in the BEM-underground directory. If an import fails, confirm that commands use .conda-env\python.exe and that the required packages were installed. If a DAT does not appear in the Example list, use Browse DAT and check its folder and extension. If a run reports an unsupported DOMAIN or loading method, consult Documentation\TRANSLATION_GAPS.md before editing the deck.

Keep DAT inputs separate from generated NPZ and PDF results. Give new runs distinct output names so reference archives are not overwritten. After changing a DAT mesh, recheck the NODE.PLOTS selections and geometry drawing. Preserve the source DAT and note the exact deck, DOMAIN, environment, and result filename when comparing LD, TD, CQ, and thesis figures.

17 Quick start checklist

- Open PowerShell in BEM-underground and check .conda-env\python.exe --version.
- If the environment is absent, install Conda and create it from environment.yml.
- Start RUN_BEM_GUI.cmd and choose a maintained DAT in examples\in.
- Inspect the deck's DOMAIN, materials, mesh, time settings, and selected nodes.
- Run the case, save a uniquely named NPZ, and compare results with the validation notes.